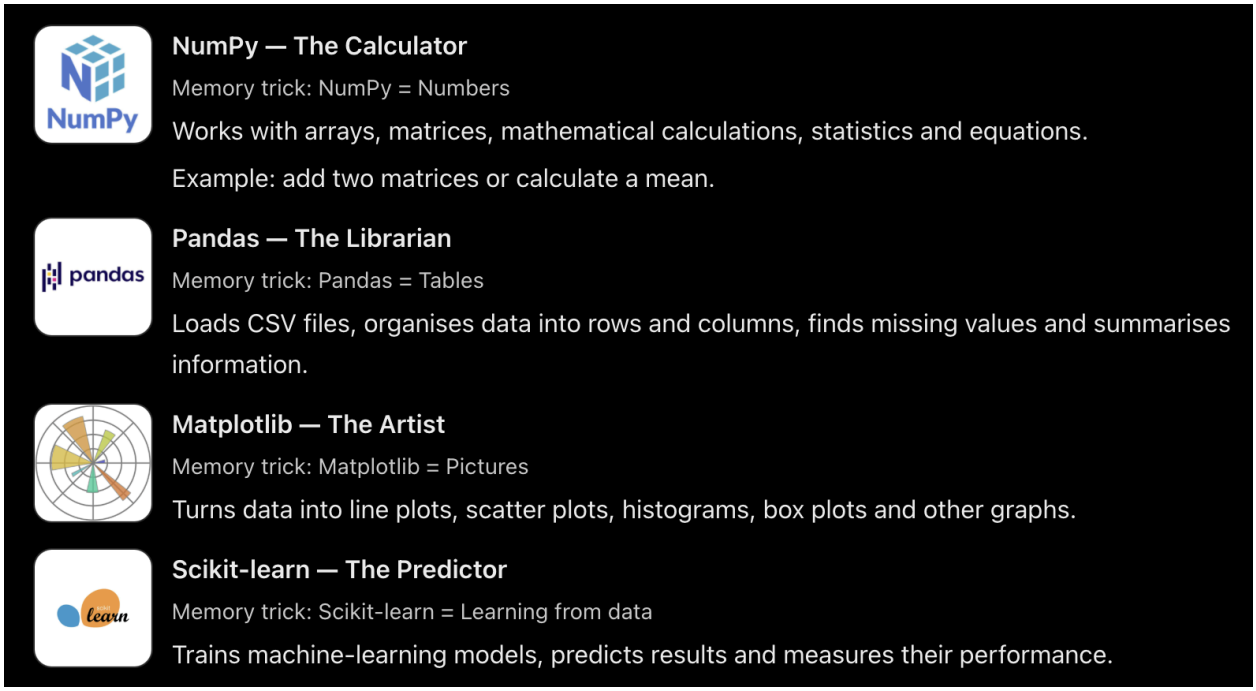
DATA SCIENCE;Notes

What is data? Data is any set of characters that is gathered and translated for some purpose, usually analysis. It can be any character, including text and numbers, pictures, sound, or video. If data is not put into context, it doesn't give any sense to a human or computer.

To make sense from data we need to work on the data using different tools.The work flow of data analysis, data science or machine learning starts from data. Data can be provided from some data source or it can be created. There are structured and and unstructure data.



NumPy

NumPy is used to work with arrays, numbers

The array object in NumPy is called ndarray.

We can create a NumPy ndarray object by using the array() function.

To create an ndarray, we can pass a list, tuple or any array-like object into the array() method, and it will be converted into an ndarray

Code	Output	Explanation
import numpy as np	-	#imported the NumPy library as np
np.__version__	1.26.4	Check version
print(dir(np))	['ALLOW_THREADS', 'BUFSIZE', 'CLIP', 'DataSource', 'ERR_CALL', 'ERR_DEFAULT', 'ERR_IGNORE', 'ERR_LOG', 'ERR_PRINT', 'ERR_RAISE', 'ERR_WARN', 'FLOATING_POINT_SUPPORT', 'FPE_DIVIDEBYZERO', 'FPE_INVALID', 'FPE_OVERFLOW', 'FPE_UNDERFLOW', 'False_', 'Inf'.....]	Checking available methods
arr = np.arange(6)	[0 1 2 3 4 5]	Creating an array with a range of values
print(arr[2]+arr[3])	5	Numpy array indexing
arr.reshape((3, 4))	[[0 1 2] [3 4 5]]	Reshaping arr to a 2D array
b = np.zeros((2, 3))	[[0. 0. 0.] [0. 0. 0.]]	Create 2 rows and 3 columns of zeros
c= np.ones((2, 3))	[[1. 1. 1.] [1. 1. 1.]]	Create 2 rows and 3 columns of ones
arr = np.array([1, 2, 3, 4, 5])	[1 2 3 4 5]	Creating an array from a list
print(type(arr))	<class 'numpy.ndarray'>	type() tells type of the object `arr` is `numpy.ndarray` type.

np.mean(arrayname)		To calculate mean of an array
np.median(array)		To calculate median of an array
np.std(array)		To calculate standard deviation of an array
np.min(array)		To find the minimum value
np.max(array)		To find the maximum value
np.dot(matrix1,matrix2)		Matrix multiplication
np.transpose(matrix1)		Transpose of a matrix
np.trace(matrix1)		Sum of diagonal elements of a matrix
a=np.power(matrix1,2)		display the elements of the matrix1 to the power of 2.
X = np.linalg.solve(A, b)		A fn in np used to solve a system of linear equation of the form AX= b

Program 1

```
import numpy as np

a = np.array([1, 2, 3])

b = np.array([4, 5, 6])


print(np.add(a, b)) #[5 7 9], Add matching positions

print(np.subtract(a, b)) #[-3 -3 -3], First minus second

print(np.multiply(a, b)) #[4 10 18], Multiply matching positions, not matrix multiplication

print(np.divide(a, b))#[0.25 0.4 0.5], Divide matching positions.


matrix1 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
matrix2 = np.array([[9, 8, 7], [6, 5, 4], [3, 2, 1]])
```

```
matrix_multiply = np.dot(matrix1, matrix2)
matrix1_transpose = np.transpose(matrix1)
diagonal_sum = np.trace(matrix1)
```

#Slicing of arrays

Slicing in python means taking elements from one given index to another given index.
We pass a slice instead of an index like this: `[start:end]`.
We can also define the step, like this: `[start:end:step]`.
If we don't pass start its considered 0
If we don't pass end its considered length of array in that dimension
If we don't pass step its considered 1

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5])    #[2,3,4,5]
print(arr[1:5:2])  #[1,3]
print(arr[-3,-1])  #[5,6]


array_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
# Slicing a subarray (rows 1 to 2, columns 0 to 1)
subarray = array_2d[1:3, 0:2]  #indexes of row,indexes of column
```

2D Array:

```
 [1  2  3]
[4  5  6]
[7  8  9]
Sliced subarray:
 [[4  5]
 [7  8]]
```

Pandas

Pandas is an open source, high-performance, easy-to-use data structures and data analysis tools for the Python programming language.
Pandas is a Python library used for working with data sets.
It has functions for analyzing, cleaning, exploring, and manipulating data.
Pandas allow us to analyze big data and make conclusions based on statistical theories.
Pandas can clean messy data sets, and make them readable and relevant.

Pandas datastructure is based on Series and DataFrames

1. A pandas Series is like a column in a table.
arr=np.array([10,15,18,22])
s = pd.Series(arr) #convert array into a series**.**
Output:

```
0    10
1    15
2    18
3    22
dtype: int64
```

2. A Pandas DataFrame is a *multidimensional table* made up of collection of *Series*

data = {'Name':['Tom', 'nick', 'krish', 'jack'],'Age':[20, 21, 19, 18]}
df = pd.DataFrame(data) # Convert the dictionary into DataFrame.

Output

```
    Name   Age
0   Tom    20
1   nick   21
2   krish  19
3   jack   18
```

(can specify the needed columns in a [[ ]] eg:(df[['Name', 'Qualification']])

Code	**Output**	**Comment**
import pandas as pd		Imported pandas library
pd.Series(50, index=[1, 2, 3, 4])	1 50 2 50 3 50 4 50 dtype: int64	A column with custom constant value and a custom index

data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'], 'Age':[27, 24, 22, 32], 'Address':['Delhi', 'Kanpur', 'Allahabad', 'Kannauj'], 'Qualification':['Msc', 'MA', 'MCA', 'Phd']} df = pd.DataFrame(data) print(df[['Name', 'Qualification']])	Name Qualification 0 Jai Msc 1 Princi MA 2 Gaurav MCA 3 Anuj Phd	A table with two custom columns
data=pd.read_csv("/content/empl.csv")		Reading a csv file
data.head()		Read first 5 rows only of the csv file
data.tail()		Read last 5 rows only
print(data.shape)	(10000,3)	Returns number of rows and columns in the csv file
print(data.isnull().sum())		Checks for missing values
print(data.columns)	`Index(['Gender', 'Height', 'Weight'], dtype='object')`	Get all the columns in the file

print(heights.describe())	`count 10000.000000`
	`mean 66.367560`
	`std 3.847528`
	`min 54.263133`
	`25% 63.505620`
	`50% 66.318070`
	`75% 69.174262`
	`max 78.998742`
	`Name: Height, dtype: float64`
		Describe() -Descriptive statistics (mean, standard deviation, number of observations, min, max, and quartiles) of numerical columns can be calculated using the describe() method, which returns a pandas dataframe of descriptive statistics.

data.describe()		describe can also give statistical information from a dataFrame

Matplotlib

Matplotlib is a low level graph plotting library in python that serves as a visualization utility.
matplotlib is a foundational plotting library for Python that provides extensive

import matplotlib as plt	Importing the plotting library
plt.plot(x, y)	Create line plot
plt.bar(categories, values)	Create bar plot ,two list variables for x and y axis
plt.hist(data, bins=30, color='blue')	Histogram, bins= no of bars(intervals), color= color of bars
plt.scatter(x, y, color='red')	Create a scatter plot of color red
plt.boxplot(data)	Creates a boxplot
plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', shadow=True, startangle=140) plt.axis('equal')	'Equal' aspect ratio ensures pie is circle
plt.title('Simple plot') plt.xlabel('Data') plt.ylabel('Values') plt.show()	Title, x axis and y axis labels, Show the plot

sizes = [100, 200, 300, 400, 500] # Bubble sizes
plt.scatter(x, y, s=sizes, alpha=0.5, c='blue')
	Create a bubble chart

control over the appearance of plots.
It's a general-purpose plotting library and is widely used for creating static, animated, and interactive visualizations
seaborn is a statistical data visualization library built on top of **matplotlib**.
It provides a high-level interface for creating attractive and informative statistical graphics with simpler syntax.




```
sns.heatmap(
    corr,
    annot=True,
    cmap='coolwarm',
    fmt='.2f',
    linewidths=0.5
)

plt.title("Correlation Heatmap", fontsize=15)
plt.show()
```

`annot=True` : Displays numbers inside cells

`Cmap` : Chooses colour scheme

`fmt='.2f'` : Displays two decimal places

`Linewidth` : Space/border thickness between cells

`cbar=True` : Displays colour scale bar

`vmin`, `vmax` : Set colour scale limits

#Bubble chart

```
import matplotlib.pyplot as plt
# Sample data with an additional dimension for colors
x = [1, 2, 3, 4, 5]
y = [10, 15, 13, 8, 7]
sizes = [100, 200, 300, 400, 500]  #bubble size
colors = [20, 30, 40, 50, 60] # Color dimension
# Creating a bubble chart with varying colors
```

plt.scatter(x, y, s=sizes, c=colors, alpha=0.6, cmap='viridis')
plt.colorbar(label='Color scale')# Adding a color bar
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Bubble Chart with Color Dimension')
plt.show()

Subplots

```
import matplotlib.pyplot as plt
import numpy as np
# Sample data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
# Create a figure with 2 subplots (vertically stacked)
fig, axs = plt.subplots(2, 1, figsize=(8, 8))
# First subplot
axs[0].plot(x, y1, 'b')
axs[0].set_title('Sine Function')
axs[0].set_ylabel('sin(x)')
# Second subplot
axs[1].plot(x, y2, 'r')
axs[1].set_title('Cosine Function')
axs[1].set_ylabel('cos(x)')
axs[1].set_xlabel('x')
plt.tight_layout()# Adjust layout to prevent overlap
plot.show()
```

Pie Chart

```
import matplotlib.pyplot as plt
sizes = [15, 30, 45, 10]
labels = ['A', 'B', 'C', 'D']
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', shadow=True,
startangle=140)
plt.title('Pie Chart')
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.
```

plt.show()

Box Plot

```
import matplotlib.pyplot as plt
import numpy as np
# Sample data
data = [np.random.normal(0, std, 100) for std in range(1, 4)]
# np.random.normal(0, std, 100) - np.random.normal(mean, std deviation,size)
# for std in range(1, 4) – list comprehension that iterates over a range of values ie, 1,2,3
plt.boxplot(data) # Create a box plot
# Add labels and title
plt.xlabel('Data Sets')
plt.ylabel('Values')
plt.title('Simple Box Plot')
# Show the plot
plt.show()
```

KNN

Programm11

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
```

`train_test_split:` Splits training and testing data

`KNeighborsClassifier:` Performs K-NN classification

`metrics:` Calculates accuracy and other metrics

```
data = load_breast_cancer()
```

```
df = pd.DataFrame(data.data, columns=data.feature_names)

df['target'] = data.target
```

`data.data` contains the actual numerical measurements.

`data.feature_names` contains their column names.

`pd.DataFrame()` converts them into a table.

Target -Adds the answer column to the DataFrame.

```
print(df.head(5))
df.info()
print(df.describe())
print(df.isnull().sum())
print(df['target'].value_counts())
```

Perform EDA

`value_counts()`:Number of records in each target class;It tells us how many cancer and non cancerous observations exist.

```
plt.figure(figsize=(8, 6))

sns.scatterplot(
    x='mean radius',
    y='mean texture',
    hue='target',
    data=df
)

plt.title("Scatter plot of Mean Radius vs Mean Texture")
plt.show()
```

- `plt.figure()` creates a plotting area.
- `figsize=(8,6)` sets the graph dimensions.
- `x` is the horizontal axis.
- `y` is the vertical axis.
- `hue` gives different colours to different target classes.
- `data=df` specifies the DataFrame.

- `title()` sets the heading.
- `show()` displays the graph.

```
X = df.drop('target', axis=1)
y = df['target']
```

X is the evidence; y is the answer.

Axis= 1 because we want to remove a column not a row.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

`Splitting the dataset`

`test_size=0.3` => means 70% training 30% testing

```
k_values = [3, 5, 7, 9] #to test 4 diff neighbour values

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = metrics.accuracy_score(y_test, y_pred)

    print(f"Accuracy with k={k}: {accuracy:.4f}")
```

Create → Fit → Predict

1. `KNeighborsClassifier(n_neighbors=5)` Creates a K-NN model with the current K.
2. `model.fit(X_train, y_train)` lets it learn from the training data.trains the model
3. `model.predict(X_test)` predicts the classes of the test data.
4. Accuracy = Compares predictions against actual answers.
5. `{accuracy:.4f}` displays accuracy to four decimal places.

CHOOSE THE HIGHEST ACCURACY OUT OF THE OUTPUT.

If K is too small, the model may be overly influenced by individual observations. If K is too large, the vote may include too many distant observations.

One extra exam point: K-NN is distance-based, so feature scaling is often important. A common pattern is `StandardScaler` inside a pipeline.

train_test_split	splitting
knn=KNeighborsClassifier(n_neighbors=k)	Creating a model
knn.fit(X_train, y_train)	fit/ learns/ trains
y_pred = knn.predict(X_test)	Predict test data
metrics.accuracy_score(y_test, y_pred)	Accuracy of diff k

Standardization (Feature Scaling).

```
# Remove missing values
df_cleaned = df.dropna()
# Features and target
X = df_cleaned.drop(['quality', 'Id'], axis=1)
y = df_cleaned['quality']
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Training learns the scale. Testing borrows the scale.

KNN

read/load a dataset -> get info, EDA -> visualize ->
Clean -> X,y -> split dataset -> standardize ->
k values -> create a model -> train -> predict ->accuracy score

The K-Nearest Neighbours algorithm was successfully applied to classify the dataset. The model predicted the test data based on the nearest neighbours. The accuracy score indicates how correctly the model classified

the observations. Different values of K can be compared to identify a suitable value.

Naive bayes

Naive Bayes = Probability + Prediction.

```
from sklearn.naive_bayes import GaussianNB

nb = GaussianNB()  #creates the model
nb.fit(X_train, y_train) #trains it
y_pred = nb.predict(X_test)  #predicts the classes
```

Programm13 & 14

```
libraries
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report, accuracy_score

df_cleaned = df.dropna() #removes rows containing missing values.

X = df_cleaned.drop(['diagnosis', 'id'], axis=1)
y = df_cleaned['diagnosis']
```

Split the dataset
Create a Naive Bayes model
Train data
Predict class

```
print(classification_report(y_test, y_pred))
print(accuracy_score(y_test, y_pred))
```

Classification report gives precision, recall, F1-score and support.
Accuracy score gives overall accuracy

The Naive Bayes classifier was trained and tested successfully. It predicted the class labels based on probability. The classification report and accuracy score were used to evaluate its performance.

Programm14

In this program the dataset has quality scores from 3 to 9.Thus

```
df['quality_label'] = df['quality'].apply(
   lambda x: 1 if x >= 7 else 0
)
```

Quality 7 or above → `1` (Good)

Quality below 7 → `0` (Not good)

Simple imputer

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
X_train = imputer.fit_transform(X_train)
X_test = imputer.transform(X_test)
```

Some wine rows have missing values. Instead of deleting them, this fills missing numerical values with the column's mean.

graphs

```
sns.scatterplot(x='radius_mean', y='texture_mean', hue='diagnosis', data=df)
df['radius_mean'].hist(bins=30)
sns.boxplot(x='diagnosis', y='radius_mean', data=df)
sns.heatmap(confusionmatrix, annot = True, fmt =’d’, cmap = ‘Blues, cbar= False)
```

LINEAR REGRESSION

Regression program sequence

1. **IMPORT – Libraries**
2. **LOAD – Read CSV**
3. **SEPARATE – X and y**
4. **SPLIT – Train and test**
5. **CREATE – `LinearRegression()`**
6. **TRAIN – `fit()`**
7. **`Intercept ,slope`**
8. **PREDICT – `predict()`**
9. **EVALUATE – Error metrics**

Libraries

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Select X&y

```
X = student.iloc[:, :-1]
y = student.iloc[:, 1]
```

Split the dataset

```
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Train the model

```
regressor = LinearRegression()
regressor.fit(x_train, y_train)
```

Intercept and coefficient

```
print("INTERCEPT =", regressor.intercept_)
print("COEFFICIENT =", regressor.coef_[0])
```

Prediction

```
y_pred = regressor.predict(x_test)

for actual, predicted in zip(y_test, y_pred):
    print("Actual:", actual, "Predicted:", predicted)

# zip() pairs each actual score with its predicted score.
```

Evaluate

```
mislabeled_points = np.sum(
    np.round(y_test) != np.round(y_pred)
)

print("Number of mislabeled points:", mislabeled_points)
#"Mislabeled points" is not really the correct term for regression; it is counting rounded prediction mismatches.Not classification errors.
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R-squared:", r2)


mae = metrics.mean_absolute_error(y_test, y_pred)
mse = metrics.mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
```

For all three, lower is better.

Linear Regression was applied to predict the dependent variable using the independent variable(s). The model's performance was evaluated using Mean Squared Error and R-squared score. A lower MSE and an R-squared value closer to 1 generally indicate better performance.

KMEANS (programm20,21)

```
from sklearn.cluster import KMeans
X = df.select_dtypes(include="number")#Selects only numerical features
X = X.fillna(X.mean())  #replace missing values with average of each column

if "Species" in df.columns:
    sns.scatterplot(
        x=X.iloc[:, 0],
        y=X.iloc[:, 1],
        hue=df["Species"],
        palette="viridis"
    )
else:
    plt.scatter(X.iloc[:, 0], X.iloc[:, 1])
```

`X.iloc[:, 0]` = first numerical feature.
`X.iloc[:, 1]` = second numerical feature.
`hue=df["Species"]` = colour points according to their actual species.
`palette="viridis"` = colour scheme.

KMEANS

```
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
```

`n_clusters=3`: Create 3 groups.
`random_state=42`: Makes the initialization reproducible.

`n_init=10`: Runs the algorithm with 10 different initial centroid placements and keeps the best result.

```
df["Cluster"] = kmeans.fit_predict(X)
```

`fit` = learns the clusters from X.

`predict` = assigns each row a cluster number.

`fit_predict` = does both together.

Stores the resulting cluster numbers in a new column called `Cluster`.

```
centroids = pd.DataFrame(
    kmeans.cluster_centers_,
    columns=X.columns
)
print(centroids)
```

- `cluster_centers_` gives the centre of each cluster.
- `pd.DataFrame()` converts the centroids into a readable table.
- `columns=X.columns` gives the correct feature names.

Centroid = average position of all points in a cluster.

```
plt.scatter(
    X.iloc[:, 0],
    X.iloc[:, 1],
    c=df["Cluster"],
    cmap="viridis",
    s=50
)
```

Plots the data points.

- `c=df["Cluster"]` = colours according to the assigned cluster.
- `cmap="viridis"` = colour scheme.
- `s=50` = point size.

```
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    color="red",
    marker="X",
    s=200,
    label="Centroids"
)
```

For K-Means, remember:

- WCSS / inertia measures within-cluster spread.
- Lower WCSS generally means points are closer to their centroids.
- The Elbow Method helps choose K.
- Don't use classification accuracy unless you have meaningful ground-truth labels and an appropriate comparison method.

Elbow Method: When K increases, WCSS generally decreases. Look for the point where the decrease starts becoming less dramatic. That bend is the elbow.

Metric	**Meaning**
Accuracy	Overall percentage of correct predictions
Precision	Of the predicted positives, how many were actually positive?
Recall	Of the actual positives, how many were identified?
F1-score	Balance between precision and recall

Confusion matrix	Shows correct and incorrect predictions by class

Metric	**Meaning**	**Better result**
MAE	Average absolute error	Lower
MSE	Average squared error	Lower
RMSE	Square root of MSE	Lower
R^2	How much variation the model explains	Generally closer to 1

Problem	**Code**	**Meaning**
Missing values	df.isnull().sum()	Find missing values
Remove missing rows	df.dropna()	Delete rows with missing values
Fill missing values	df.fillna(df.mean(numeric_only=True))	Fill numerical missing values with means
Remove duplicates	df.drop_duplicates()	Remove repeated rows
Remove a column	df.drop('ID', axis=1)	Delete a column
Select numerical data	df.select_dtypes(include='number')	Keep numerical columns
Select text columns	df.select_dtypes(include='object')	Keep text columns

Model	Scaling generally needed?
K-NN	Yes
Naive Bayes (GaussianNB)	Usually not necessary
Decision Tree	No
Linear Regression	Not usually necessary for ordinary linear regression
K-Means	Yes, generally

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Answer format

Section	What you should do
1. Aim	State what the experiment performs
2. Import	Import required libraries
3. Load	Read the given dataset
4. EDA	Display head, shape, info, describe
5. Cleaning	Handle missing values and unnecessary columns
6. Visualization	Create the required plots
7. Feature selection	Select X and y, if supervised
8. Preprocessing	Split and scale if needed

9. Model	Create the correct algorithm
10. Training	Use fit() or fit_predict()
11. Prediction	Use predict() if supervised
12. Evaluation	Display suitable metrics
13. Conclusion	Explain what the model achieved

Code	**Meaning**
df.head()	First five rows
df.tail()	Last five rows
df.shape	Dataset dimensions
df.columns	Column names
df.info()	Data types and non-null counts
df.describe()	Statistical summary
df.dtypes	Data types
df.isnull().sum()	Missing values
df.dropna()	Remove missing rows
df.fillna()	Fill missing values
df.drop_duplicates()	Remove duplicates
df['Marks']	Select one column
df[['Age','Marks']]	Select multiple columns

df.iloc[:, :-1]	All rows, all but last column
df.iloc[:, -1]	Last column
df.value_counts()	Count category frequencies
df.nunique()	Count unique values
plt.show()	Display plot
plt.figure()	Create figure
plt.scatter()	Scatter plot
plt.hist()	Histogram
plt.boxplot()	Boxplot

Function	**What it does**	**Explanation**
pd.read_csv()	Reads a CSV file	Used to load a dataset into a DataFrame.
pd.read_csv(..., sep=';')	Reads a semicolon-separated CSV	sep specifies the separator used in the file.
head()	Displays first 5 rows by default	Used to preview the dataset.
tail()	Displays last 5 rows by default	Used to view the last records.
shape	Gives rows and columns	Returns a tuple: (rows, columns).

info()	Shows column names, data types and non-null counts	Used to understand the dataset structure.
describe()	Gives statistical summary	Shows count, mean, standard deviation, minimum, maximum and quartiles for numerical columns.
columns	Shows column names	Used to inspect available variables.
isnull()	Identifies missing values	Returns True for missing entries.
sum()	Adds values	With isnull(), counts missing values in each column.
duplicated()	Checks duplicate rows	Returns True for rows that are duplicates of earlier rows.
value_counts()	Counts each unique value	Useful for checking class distribution or cluster sizes.

B. Selecting and cleaning data

Function	**What it does**	**Explanation**
drop()	Removes rows or columns	Used to remove unwanted columns such as ID or the target from X.
dropna()	Removes rows with missing values	Used when you choose to discard incomplete records.
fillna()	Replaces missing values	Used to fill missing entries with a value such as the mean.
mean()	Calculates average	Can be used to fill missing numerical values.
select_dtypes()	Selects columns by data type	include='number' selects numerical columns.

nunique()	Counts unique values	Can identify constant columns that have only one unique value.
sort_index()	Sorts by index	In your Iris program, it sorts cluster counts by cluster number.
copy()	Creates a copy of a DataFrame	Helps avoid unintentionally modifying the original data.
iloc[]	Selects by integer position	iloc[:, :-1] selects all rows and all columns except the last.
loc[]	Selects by labels	Used to select rows or columns by their labels.
apply()	Applies a function to values	Used to convert wine quality scores into 0 and 1 labels.
lambda	Creates a small anonymous function	Used for short operations inside apply().

C. Data splitting and preprocessing

Function	**What it does**	**Explanation**
train_test_split()	Splits data into training and testing sets	Training data builds the model; test data evaluates it.
StandardScaler()	Creates a standardisation tool	Standardises numerical features to a comparable scale.
fit()	Learns from data	For a model, it trains the model; for a scaler, it learns the mean and standard deviation.
transform()	Applies a learned transformation	Uses the scaler's learned values to standardise data.

fit_transform()	Learns and transforms in one step	Commonly used to fit a scaler and transform training data.
SimpleImputer()	Creates a missing-value replacement tool	Can replace missing numerical values with the mean.

-

D. Machine-learning model functions

Function	**Algorithm**	**Meaning**
KNeighborsClassifier()	K-NN	Creates a nearest-neighbour classification model.
GaussianNB()	Naive Bayes	Creates a Gaussian Naive Bayes model.
DecisionTreeClassifier()	Decision Tree	Creates a decision-tree classification model.
KMeans()	K-Means	Creates a clustering model.
LinearRegression()	Linear Regression	Creates a linear regression model.
fit(X_train, y_train)	Supervised learning	Trains a model using features and known answers.
predict(X_test)	Supervised learning	Predicts answers for test features.
fit_predict(X)	Clustering	Learns clusters and returns the cluster assigned to each row.

E. Evaluation functions

Function	**Used for**	**Meaning**

accuracy_score()	Classification	Proportion of predictions that are correct.
classification_report()	Classification	Displays precision, recall, F1-score and support.
confusion_matrix()	Classification	Shows correct and incorrect predictions by class.
mean_squared_error()	Regression	Calculates average squared prediction error.
mean_absolute_error()	Regression	Calculates average absolute prediction error.
r2_score()	Regression	Measures how well the model explains variation in the target.
inertia_	K-Means	Gives the within-cluster sum of squared distances (WCSS).
cluster_centers_	K-Means	Gives the coordinates of cluster centres.
n_clusters	K-Means	Stores the number of clusters requested.

F. Plotting and visualisation functions

Function	**What it does**	**When you use it**
plt.figure()	Creates a new figure	Before making a plot
plt.scatter()	Creates a scatter plot	Shows relationship between two numerical variables
plt.hist()	Creates a histogram	Shows the frequency distribution of a variable
plt.plot()	Creates a line plot	Used for the Elbow Method

plt.xlabel()	Labels the x-axis	Names the horizontal axis
plt.ylabel()	Labels the y-axis	Names the vertical axis
plt.title()	Adds a title	Describes the graph
plt.legend()	Displays a legend	Explains colours or symbols
plt.show()	Displays the plot	Shows the graph output
plt.scatter(..., c=clusters)	Colours points by cluster	Visualises K-Means groups
sns.scatterplot()	Seaborn scatter plot	Can colour points using a category with hue
sns.boxplot()	Creates a box plot	Shows median, spread and possible outliers
sns.heatmap()	Creates a heatmap	Visualises values such as a confusion matrix
plot_tree()	Draws a decision tree	Shows the tree's decision rules

Function / expression	**Meaning**
np.array()	Creates a NumPy array.
np.arange()	Creates a sequence of numbers with a specified step.
np.zeros()	Creates an array filled with zeros.
np.ones()	Creates an array filled with ones.
np.reshape()	Changes the shape of an array without changing its data.

np.mean()	Calculates the mean.
np.median()	Calculates the median.
np.std()	Calculates standard deviation.
np.min() / np.max()	Finds minimum / maximum values.
np.sum()	Adds array values.
len()	Returns the number of items.
range()	Generates a sequence of integers.
print()	Displays output.
type()	Returns the data type of an object.
int() / float() / str()	Converts values into different data types.
input()	Takes input from the user.